INCUBRIX ENGINEERING ASSESSMENT 1

AI Background Music Engine for Video

Original, commercially safe instrumental music that fits a video exactly - with real instrument stems

Thank you for applying to IncuBrix

IncuBrix helps creators make videos with AI. In this assessment you will build one real piece of our product. Please read the whole brief once before you start; it explains exactly what we want and exactly how we will score it.

At a glance	
What you build	A service that turns a mood description and a video's timing into an original instrumental music track, delivered as a finished mix plus separate instrument stems.
Why it matters	Every IncuBrix video needs music. Stock music repeats, rarely fits the cut and can trigger copyright claims. Generated music that fits exactly and is safe to monetise is a core product feature.
What you hand over	A containerised API service, a live demo URL, your code, a short report and a test pack.
Time allowed	10 calendar days from the day you receive this brief. The exact deadline is in the email.
Tools allowed	Anything: any AI coding assistant, model, API, cloud or library. Just disclose it and keep it commercially usable.
How you are selected	You fill in a Submission Form showing what works, with proof. We verify it and run 5 hidden tests (screening, 80 marks), then hold a 60-90 minute live session (20 marks) with the strongest candidates. Shortlist: 75+. Top pick: 85+.

How to use this brief

1. Read **The problem** and **What you build** to understand the job.
2. Build the **Must-haves** first. Each one is pass or fail, and every one must pass.
3. Then climb the **Levels**: Core is required; Advanced and Outstanding earn more marks.
4. Follow the **Integration Standard** so your work can plug straight into our product.
5. Fill in the **Submission Form** (Excel file sent with this brief) as you go. It is how you show us your work, and it is the main thing we review.

The problem

A creator finishes a 45-second product video. It has a calm intro, a build-up and an exciting finish, and a voiceover runs on top. Today they search a stock library, find a track that is 2 minutes 37 seconds long, cut it awkwardly, and lower its volume by hand wherever someone speaks. The music often clashes with the edit points, and the same track appears in thousands of other videos.

We want music that is **made for this video**: the right mood, exactly the right length, changing at the right moments, and delivered as **separate instrument tracks (stems)** so the IncuBrix editor can rebalance it, swap the drums, or lower it under the voiceover automatically.

The hard part

Most AI music generators produce one mixed audio file. A mixed file is not the same as separate instrument parts. Your system must deliver stems that are genuinely separate instrument parts which add back up to the final mix - not the same mix split into frequency bands with an equaliser. How you achieve that is your engineering decision, and your report must explain it.

What you build

Input

A JSON request. Only `prompt` and `duration_s` are required; your system must handle every other field when present.

```
POST /api/v1/music/jobs
{
  "prompt": "warm, optimistic acoustic pop for a product launch, builds to an uplifting finish",
  "duration_s": 45.0,                       // 5 to 300 seconds
  "sections": [                             // optional: mood changes at video cut points
    {"start_s": 0.0,  "mood": "calm intro"},
    {"start_s": 12.5, "mood": "build-up"},
    {"start_s": 30.0, "mood": "uplifting peak"}
  ],
  "tempo_bpm": null, "key": null,           // optional: fix tempo or key
  "loopable": false,                        // optional: must loop seamlessly
  "narration_url": "https://.../voiceover.wav",  // optional: lower the music under speech
  "seed": 42,
  "output": {"sample_rate": 48000, "target_lufs": -16, "formats": ["wav", "mp3"]}
}
```

Output (returned as download links when the job succeeds)

File	Contents
mix.wav (and mix.mp3)	The finished instrumental track, exactly the requested length.
stems/*.wav	At least 3 separate instrument parts, for example drums, bass, harmony/pads and melody. Same length and sample rate as the mix.
mix_ducked.wav	Only when narration is given: the mix lowered under speech (the voiceover itself is not included).
music_manifest.json	Tempo, key, section times, beat and bar times, the gain of each stem in the mix, how each stem was produced, models/providers and versions, seed, loudness, timings, cost, and licence/rights for every component.

Must-haves (all must pass)

#	Must-have	How we verify it (you give the evidence in your form)
1	**Exact length.** The mix and every stem match the requested duration within 0.05 s, and the track ends on purpose (a final hit or a fade on a bar line), never a hard cut.	We measure the file lengths and listen to the last 3 seconds of 10 outputs.
2	**Real stems.** At least 3 stems, each one a different instrument part. The mix equals the sum of the stems using the gains in the manifest.	We add the stems back together and compare with the mix, then play every stem on its own. A stem that is a frequency band of the full mix fails.
3	**Instrumental only.** No vocals, singing or spoken words.	Listening check on all outputs.
4	**Safe to publish commercially.** Every model, service, sample and loop in the default path allows commercial use of its output, with a link to the terms in SOURCES.md. The system never copies an existing song.	We read SOURCES.md and the terms, and test prompts that name real songs and artists.
5	**Ready-to-use audio.** 48 kHz WAV; loudness at the target (default -16 LUFS) within 1 LU; true peak at or below -1 dBTP; no clipping, clicks or gaps at section changes.	We measure loudness and peaks with a standard meter and listen at every section change.
6	**Integration Standard** followed and the **live demo** works for the whole review period.	See the Integration Standard section.

Levels

Level	What you need to show
Core (required)	• All must-haves pass. • 10 clearly different prompts (for example cinematic, lo-fi, corporate, electronic, acoustic, tense, playful) each produce music that obviously matches the description. • Durations from 10 seconds to 3 minutes all work.
Advanced	• **Follows the section plan:** the energy or instrumentation changes within 0.5 s of each section start, and changes land on a beat. • **One musical piece:** all stems share the same tempo and key; the beat and bar grid is in the manifest. • **Voiceover ducking:** with narration, the music drops 8-15 dB under speech and comes back smoothly in gaps, with no pumping. The volume curve is in the manifest. • **Change one part:** regenerate one stem (for example new drums) without changing the others. • **Seamless loop** when `loopable` is true: no click and no break in the beat at the loop point.
Outstanding	• **Measured quality:** prompt match scored with an independent audio-text model (for example CLAP), not the generator scoring itself; stem cleanliness (bleed between stems) measured; a blind listening test with at least 5 people against a simple baseline. • **Two approaches compared** with numbers (quality, speed, cost per minute of music), and a clear recommendation of which one IncuBrix should run in production and why. • **Production numbers:** time and cost to generate 1 minute of music, and how many jobs one server handles at once.

Hard cases we will test

Build and test your system against situations like these, and show the results in the Hard cases tab of your Submission Form. During screening we run **5 hidden tests** of the same kinds on your live demo. A good system handles each one sensibly and says what it did (in warnings or error messages) instead of failing silently.

- Very short (5-6 seconds) and long (5 minutes) tracks.
- Vague or contradictory prompts, such as "calm but aggressive".
- Prompts that name a real song or artist, or ask for vocals. The system must not copy the song and must stay instrumental, and should say so.
- Section changes very close together (under 1.5 seconds apart) or a section starting after the end of the track.
- Narration with long silences, or no speech at all.
- A prompt in a language other than English.
- Two jobs sent at the same moment, and the same job sent twice with the same Idempotency-Key.
- Your music provider or model failing or timing out in the middle of a job.

Helpful pointers (not requirements)

- Possible approaches include: a text-to-music model followed by a source-separation model; generating each instrument separately with a shared tempo, key and bar grid; symbolic (MIDI) composition rendered with licensed virtual instruments; or a hosted music API that returns stems. Each has trade-offs in quality, stem cleanliness, exact-length control and cost. Explain yours.
- Exact length is easier if you plan in bars: choose a tempo so the sections land on bar lines, then render a musical ending.
- Parts generated separately will not automatically agree on tempo, key or phrasing. If you go that way, show how you keep them together.
- Measure, do not assume. A short table of numbers beats a page of adjectives.

How it must plug into IncuBrix (Integration Standard)

IncuBrix runs its services as containers that talk to each other over HTTP APIs. If your submission follows the ten points below, we can connect it to the product without rewriting it. Each point is worth 2 marks (20 marks in total). Tick each one in your Submission Form with a link to the proof.

Requirement	What exactly we check
1. One-command start and test	`docker compose up` starts the service and everything it needs (database, storage, local test servers). One command runs all tests (unit, integration and at least one end-to-end) with no API keys. A `.env.example` lists every setting.
2. Documented HTTP API	An `openapi.yaml` (OpenAPI 3.1) that matches the running service exactly. All routes under `/api/v1/`.
3. Background jobs	Anything slower than 2 seconds runs as a job: `POST` returns `202` with a `job_id`; `GET /api/v1/jobs/{job_id}` returns status (queued, running, succeeded, failed), progress 0-100 and result links. Optional callback URL, signed with HMAC-SHA256 in an `X-IncuBrix-Signature` header.
4. Safe retries	An `Idempotency-Key` header on every POST: sending the same request twice with the same key returns the same job and never does the work twice.
5. Clear errors	Errors are JSON: `{"error": {"code", "message", "retryable", "details"}}`. Codes are UPPER_SNAKE_CASE and listed in the README.
6. Configuration	All settings, paths, model names and limits come from environment variables or a config file. No hard-coded paths or keys.
7. Files in, files out	Inputs accepted as an upload or a URL. Outputs saved to S3-compatible storage (MinIO in the compose file is fine) and returned as download links.
8. Logs and health	Structured JSON logs with timestamp, level, job_id and the `X-Correlation-Id` from the request. `GET /health` and `GET /ready` endpoints.
9. Output manifest	Every result includes a JSON manifest: which models/providers and versions were used, settings, seed, timings, cost estimate, warnings, and licence/rights information.
10. Batch runner	A command-line tool that runs a folder or file of test inputs against the API and writes all results to one folder. We use this to test every submission the same way.

Tools and resources: use anything

We want to see the best system you can build, the way you would build it at work. There are no restrictions on tools.

- **AI coding assistants** (ChatGPT, Claude, Copilot, Cursor and so on): allowed, as much as you like.
- **Any AI model or service**: open-source or proprietary, self-hosted or hosted API, free or paid.
- **Any compute**: your laptop, a cloud GPU, a free notebook, a paid server.
- **Any open-source code or library**, as long as you credit it and its licence allows commercial use.

Because your solution should be able to go straight into the IncuBrix product, five rules apply:

1. **Disclose everything.** List every model, API, library, dataset and media source in SOURCES.md, and every AI coding tool you used (and for what) in AI_USE.md.
2. **Commercially usable.** Everything in your default path, and everything it outputs, must be licensed for commercial use. No research-only or non-commercial models, data or media (for example CC BY-NC) in the default path. Avoid AGPL code in the shipped service.
3. **Swappable providers.** Put every external AI service behind a small adapter selected by configuration, and include a mock adapter so tests run with no API keys.
4. **Know your cost.** Report the cost and time per unit of output (for example per minute of video). Paid services are fine, but you will not get extra marks for spending more. IncuBrix does not reimburse costs unless agreed in writing before you spend.
5. **No secrets in the code.** API keys live in environment variables or a secret store, never in the repository, logs or screenshots.

How we evaluate (simple, three steps)

You do most of the work of showing what you built: you fill in the **Submission Form** with what works and a link to the proof for each claim. We then check your claims and test your system ourselves.

Step	Who	What happens
1. Submission	You	Send the links and the completed Submission Form: must-haves met, levels reached, Integration Standard points done, numbers measured, hard cases tested. Every claim needs a link to evidence (a file, log, video timestamp, test or URL).
2. Screening	Us, about 45 minutes	We check that the must-haves really pass, verify your claims using your evidence and your live demo, and run 5 hidden tests. This gives a screening score out of 80.
3. Live session	You and us, 60-90 minutes	Only for candidates whose must-haves all pass and who score at least 60 of 80 in screening. Scored out of 20.

Marks (100)

Part	Marks	How the marks are given
Must-haves	Pass / fail	Every must-have must pass. One failure means the submission is not selected.
Levels reached	40	Core 15 + Advanced 15 + Outstanding 10. A level counts only when every item in it is verified, and only if the level before it is verified.
Integration Standard	20	2 marks for each of the 10 points we confirm.
Hidden tests	20	5 tests on your live demo, 4 marks each: 4 = handled correctly, 2 = handled with a clear flaw, 0 = wrong, crash or silent failure.
Live session	20	4 parts, 5 marks each (see the live session section).

Be precise and honest in your form

A claim without evidence scores 0. A claim that turns out to be false disqualifies the whole submission. It is far better to mark something "No" or "Partial" and explain why.

Numbers you must report (Metrics tab of the form)

Metric	How to measure it
Generation speed	Seconds to produce 60 s of music, cold and warm, on stated hardware.
Cost	Cost per minute of music, including API fees or compute.
Length accuracy	Largest difference between requested and delivered duration across all tests.
Loudness	Integrated LUFS and true peak of every mix.
Stem quality	Difference between the sum of stems and the mix (in dB), plus a bleed estimate per stem.
Prompt match (Outstanding)	Score from an independent audio-text model, plus listening-test results.

Who we select

We are hiring only the strongest engineers, so the bar is high and the same for everyone.

Result	What it takes
Top pick - hire	All must-haves pass, total of 85 or more, Outstanding level verified, and at least 14 of 20 in the live session.
Shortlist - hire	All must-haves pass, total of 75 or more, and at least 14 of 20 in the live session.
Not selected	Any must-have fails, screening below 60 of 80, total below 75, or live session below 14.
Disqualified	Any false claim in your form, fake or edited evidence, hidden manual work presented as automated, using someone else's work without credit, using people's voice, face or content without rights, or not being able to explain your own code.

If two candidates tie, the higher live-session score wins, then hidden tests, then Integration Standard.

The live session (60-90 minutes, online)

If all your must-haves pass and you score at least 60 of 80 in screening, we invite you to a live session. You share your screen and work in your own repository and environment. You may use documentation and your usual AI coding tools. Each part is scored 5 (excellent), 3 (good), 1 (weak) or 0 (not shown).

Part	What happens	What a 5 looks like
1. Walkthrough	You explain the architecture and trace one output field from the request through to the final file.	Clear, accurate, you know why each piece exists and what would break if it changed.
2. Predict and run	We give you an input you have not seen. You predict what your system will do, then run it.	Prediction matches; when it does not, you explain why within minutes.
3. Debugging	We break something (a setting, a dependency, a provider response) and you find and fix it.	You find the cause from logs and code quickly and fix it properly, not with a workaround.
4. Live change	For example: add a new stem type, a new manifest field, or a minimum-gap rule for sections.	You write or update a test first or alongside, make the change, run the tests and the end-to-end path, and it works.

What to send us

Reply to the invitation email before the deadline, attaching your completed Submission Form. Put all links in the form. Keep every link working until we announce results.

Item	What it must contain
Submission Form	The Excel file sent with this brief, fully completed: every must-have, level item and Integration Standard point marked Yes / Partial / No with a link to the evidence, plus your measured numbers and hard-case results. This is the main thing we review.
Live demo URL	A working URL plus an access token if you protect it. Tell us the cold-start time if there is one. Put a rate limit on it so it cannot be abused.
Repository	Private GitHub repository with reviewer access (the username is in the email). Real commit history, not one final commit.
README.md	What it does, architecture diagram, setup in under 10 commands, how to run tests and the batch runner, all settings.
openapi.yaml	The API contract (Integration Standard point 2).
REPORT.md	Two to four pages: the approaches you considered and why you chose yours, what works, what does not and why, measured numbers, cost per unit, and what you would do next.
SOURCES.md and AI_USE.md	As described in the tools section.

Item	What it must contain
Test pack	All inputs you tested with, all outputs (including failures), logs, and an index linking each claim in your report to a file.
Demo video	Unedited screen recording, at most 8 minutes: one clean run, one failure and how the system handles it, the tests running, and one design decision explained in code.
Audio pack	For every test: the request, the mix, every stem, the manifest and the logs. Include failed and weak results.

Rules, rights and using your work

- **Individual work.** One candidate, one submission. You may use any tools and open-source code, but the design and the core logic must be yours and you must be able to explain all of it.
- **Rights.** Use only media, audio, data and content that you own, that is public domain, or that is licensed for commercial use. Never use a real person's voice, face or content without their consent.
- **Label AI output.** Mark generated media as AI-generated in the manifest.
- **Honesty.** Keep failed attempts in your test pack. An honest report of what does not work scores better than a polished demo that hides problems.
- **Using your work in our product.** This assessment is designed so that a strong submission can go straight into IncuBrix. Your code remains yours. If we want to use it, we will agree the terms with you in writing (licence or ownership, and compensation) before any use. By submitting, you confirm that the work is yours and that it and everything it depends on can legally be used commercially.
- **Questions.** Reply to the invitation email. We answer within one working day and share useful answers with all candidates on the same assessment, so nobody gets an unfair advantage.

Quick answers

Question	Answer
Can I use ChatGPT, Claude, Copilot or similar?	Yes. Tell us how in AI_USE.md. In the live session you must be able to explain and change the code yourself.
Can I use paid APIs or a paid GPU?	Yes. Report the cost per unit. We do not reimburse costs unless agreed in writing first.
Can I start from an open-source project?	Yes, with credit and a licence that allows commercial use. We score what you added and how well it works.
Do I need a nice user interface?	No. A simple web page or the API docs page is enough. We score the engine, the integration and the results.
What if I cannot finish everything?	Make the must-haves and the Core level solid first. Then go as far up the levels as you can. Report honestly what is missing.
Can I use a hosted music API?	Yes, if its terms give IncuBrix commercial rights to the output and you can still deliver real stems, exact length and the other must-haves.

We look forward to seeing what you build. Good luck.

IncuBrix Engineering Team